

Under What Conditions Do Custom Ternary Matrix Multiplication Kernels Provide Measurable Inference Performance Advantages Over Existing Low-Bit Methods on Consumer GPUs?

Abstract

Extreme weight quantization schemes, including ternary $\{-1, 0, 1\}$ representations, replace the multiplication operation in matrix multiplication with addition, negation, and masking, motivated by the hardware cost of multiplier circuitry. Prior work has shown that ternary LLMs can match full-precision model quality, but existing kernel-level benchmarks of this advantage have focused primarily on datacenter GPUs. This paper investigates whether a custom Triton ternary matrix multiplication kernel provides a measurable inference speedup over off-the-shelf low-bit methods (INT8, INT4 via `bitsandbytes`) on a consumer GPU (NVIDIA RTX 2060), and under what conditions. Across a sweep of matrix sizes and batch sizes, we find that a tensor-core-based ternary kernel outperforms INT8 and INT4 specifically in the memory-bound regime, but loses once operations become compute-bound, a transition that occurs at substantially smaller matrix sizes as batch size grows. A multiplication-avoiding kernel variant, which does not use tensor cores, performs far worse than the tensor-core variant at every tested size, indicating that avoiding multiplication in itself does not yield a hardware benefit; rather, access to fused multiply-accumulate hardware dominates. We further find that INT4 quantization, despite its smaller nominal weight size, uses more peak memory than FP16, INT8, or ternary representations, likely due to dequantization overhead. These results suggest that the practical benefit of ternary computation on consumer GPUs is condition-dependent and implementation-sensitive, rather than a fixed property of the representation itself.

1 Introduction

Modern large language models store their parameters in 16 bit floating point (FP16), a format that treats every weight as carrying fine grained magnitude information. While FP32 was once the standard, FP16 cuts memory in half, while being more accurate. However, this still has real hardware costs, as a model of 1 billion parameters requires approximately 2GB of memory for weights alone, before accounting for activations or key-value (KV) caching during inference and every matrix multiplication. This computation requires dedicated multiplier circuitry that is expensive in terms of both silicon and energy.

Recent work on extreme quantization has proposed a radically different representation: constraining every weight to one of the three values, $\{-1, 0, 1\}$. Under this constraint, the computation of matrix multiplication degenerates into simple addition and subtraction, eliminating the need for multiplier hardware. Previous work (BitNet, BitNet b1.58) has shown that models trained under this constraint of several billion parameters can approach the quality of their full precision counterparts, an initially counter intuitive result that appears to rely on the substantial redundancy present in large neural networks. Rather than precision, the sign matters more due to the aggregate statistical pattern across thousands of weights per computation.

What remains largely unexplored is whether this theoretical hardware advantage actually materializes as measurable and impactful inference speedup in practice. Existing kernel level benchmarks of low bit

and ternary approaches usually focus on datacenter class GPUs, opening the question of how these methods work closer to our day to day lives in consumer GPUs. It is not inherently obvious that eliminating a multiplier circuit guarantees a real world speed up; a naive approach may trade one bottleneck for another. This paper investigates whether, and under which conditions (specifically at which matrix sizes), a custom ternary matrix multiplication kernel provides a measurable performance advantage over existing low bit inference methods on a consumer GPU.

This paper presents a Triton based ternary matrix multiplication kernel that implements this multiplication free computation directly, replacing multiplication with addition, negation and masking. We benchmark this kernel against dense FP16 inference and off the shelf low bit methods (INT8 and INT4 via `bitsandbytes`) across a range of matrix sizes on an RTX 2060, with the goal of identifying whether, and at what scale, the theoretical advantage of ternary computation translates into speedup, or is instead outweighed by memory bandwidth and kernel overhead.

2 Related Work

Post-training and quantization-aware methods for reducing LLM inference cost span a range of bit-widths. `LLM.int8()` [2] introduced an 8-bit matrix multiplication procedure for transformers that preserves full-precision performance by handling outlier features separately, and is the method underlying the `bitsandbytes` INT8 baseline used in this work. 4-bit weight quantization, as used in the `bitsandbytes` INT4 baseline here, was popularized alongside efficient finetuning methods such as QLoRA [3].

At the extreme end of this spectrum, BitNet [6] introduced a 1-bit Transformer architecture trained from scratch with binarized weights, showing competitive language modeling performance at reduced memory and energy cost relative to FP16 baselines. BitNet b1.58 [4] extended this to a ternary $\{-1, 0, 1\}$ weight representation, matching full-precision performance at equivalent model size and training tokens while reporting substantial improvements in latency, memory, and throughput at scale. Both works motivate the premise of this paper: that ternary weight representations can, in principle, preserve model quality while eliminating the multiplication operation from the dominant matrix multiplications in a transformer.

This paper does not evaluate model quality under ternary quantization; instead, it takes the accuracy case as established by prior work and asks a separate, kernel-level question: whether the hardware advantage implied by eliminating multiplication is realized as measured speedup on a single consumer GPU, using a custom Triton kernel rather than purpose-built or vendor-optimized infrastructure.

The kernels in this work were implemented in Triton [5], a Python-embedded language and compiler for writing custom GPU kernels without directly authoring CUDA. Quantization was applied to a pretrained model from the Pythia suite [1].

3 Method

3.1 Ternary Quantization

Weights are quantized to ternary values $\{-1, 0, 1\}$ using an absmean scaling scheme. Given a weight matrix $W \in \mathbb{R}^{m \times n}$, a single scalar scale s is computed as the mean absolute value of all elements,

$$s = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n |W_{ij}|. \quad (1)$$

Each weight is then divided by the scale, rounded to the nearest integer, and clamped to the range $[-1, 1]$:

$$\widehat{W}_{ij} = \text{clamp}\left(\text{round}\left(\frac{W_{ij}}{s}\right), -1, 1\right), \quad \widehat{W}_{ij} \in \{-1, 0, 1\}. \quad (2)$$

Absmean scaling was chosen over absmax scaling because absmax is dominated by a single outlier weight, which wastes most of the three-value range on magnitudes that occur rarely; absmean keeps the typical weight magnitude mapped near ± 1 , so a larger share of the weight matrix uses the full ternary alphabet.

The quantization scheme was applied post-hoc to a pretrained Pythia-160M checkpoint, without any retraining or quantization-aware training (QAT). This was a deliberate decision: because neither kernel correctness nor kernel performance depends on model quality, quantizing existing weights directly avoided the multi-hour training iteration cycles that proved impractical on the available hardware.

3.2 Kernel Design

Two Triton kernel variants were implemented, both computing the same underlying operation: matrix multiplication between an activation matrix X and a ternary weight matrix $\widehat{W}$, differing only in how the multiply-accumulate step is expressed. The dense reference operation is

$$Y_{mn} = s \sum_k X_{mk} \widehat{W}_{kn}. \quad (3)$$

Selection-based kernel. Because every entry of $\widehat{W}$ is constrained to $\{-1, 0, 1\}$, the contribution of each weight to the output can be expressed as a conditional selection rather than a multiplication:

$$X_{mk} \widehat{W}_{kn} = \begin{cases} +X_{mk} & \text{if } \widehat{W}_{kn} = 1, \\ 0 & \text{if } \widehat{W}_{kn} = 0, \\ -X_{mk} & \text{if } \widehat{W}_{kn} = -1. \end{cases} \quad (4)$$

This kernel implements the selection directly using `tl.where`, broadcasting the activation and weight tiles against each other and reducing over the shared dimension, without invoking any multiply instruction on the weight term (Algorithm 1). This version was used to establish functional correctness and to test the hypothesis that avoiding multiplication yields a measurable hardware benefit.

Tensor-core kernel. The second variant casts the ternary weights to `FP16` and combines them with activations using `tl.dot`, Triton’s tensor-core matrix multiply primitive. While this reintroduces multiplication, it serves as a controlled comparison: both kernels share identical tiling, masking, and accumulation structure, and differ solely in this combine step, so any performance difference is attributable to that single decision.

Both kernels accumulate internally in `FP32` regardless of input or output data type, avoiding the precision loss that comes from summing many low-precision values; the result is narrowed to the output type only at the final store.

Algorithm 1 Ternary matmul, selection-based inner loop (per output tile)

Require: activation tile $X \in \mathbb{R}^{B_M \times B_K}$, ternary weight tile $\widehat{W} \in \{-1, 0, 1\}^{B_K \times B_N}$, scale s

- 1: $\text{acc} \leftarrow \mathbf{0} \in \mathbb{R}^{B_M \times B_N}$ ▷ accumulator in FP32
- 2: **for** each K-block along the shared dimension **do**
- 3: load X tile (FP16/FP32), load $\widehat{W}$ tile (int8)
- 4: $\text{contrib} \leftarrow \text{WHERE}(\widehat{W}=1, +X, \text{WHERE}(\widehat{W}=-1, -X, 0))$
- 5: $\text{acc} \leftarrow \text{acc} + \sum_k \text{contrib}$ ▷ reduce over K
- 6: **end for**
- 7: **return** $s \cdot \text{acc}$ ▷ single scalar multiply, applied once

Correctness for both kernels was verified against a dense reference computation, $X @ (\widehat{W} \cdot s)$, computed directly in PyTorch, across multiple matrix shapes and batch sizes, including sizes not evenly divisible by the kernel’s block dimensions, to exercise boundary-masking logic rather than only the common case.

An autotuned variant of the tensor-core kernel was additionally built using `triton.autotune`, sweeping seven block-size and launch-parameter configurations. This is reported as part of the experimental investigation (Section 4.4) rather than as the primary kernel, since autotuning did not improve on the manually chosen configuration.

3.3 Experimental Setup

Four implementations were compared: dense FP16 matrix multiplication (`torch.matmul`, using cuBLAS), 8-bit and 4-bit quantized linear layers via the `bitsandbytes` library (`Linear8bitLt` and `Linear4bit`, the latter configured with FP16 compute dtype to match its FP16 input rather than the library’s FP32 default), and the tensor-core ternary kernel described above.

Two variables were swept: matrix size ($K \times N$) across 8 configurations from 128×128 to 6144×6144 , chosen to include the two real layer shapes in Pythia-160M’s MLP block alongside smaller and larger synthetic shapes; and batch size $M \in \{1, 32, 128, 256\}$, to span single-token inference through batched serving.

Times were measured using CUDA events rather than wall-clock time, since GPU operations execute asynchronously and naive CPU-side timing would primarily measure kernel launch latency rather than execution time. Each configuration was run for 10 untimed warm-up iterations, discarded to exclude one-time compilation and allocation costs, followed by 50 timed iterations that were averaged.

Peak memory usage was measured separately using `torch.cuda.max_memory_allocated()`, reset immediately before each measured call to isolate that implementation’s peak from residual allocations left by prior implementations and from the caching allocator’s retained memory between differently sized configurations.

To determine whether a configuration was compute-bound or memory-bound, a roofline analysis was performed for each run. For an $M \times K \times N$ matmul, the operation performs

$$\text{FLOPs} = 2 M K N, \quad (5)$$

and the arithmetic intensity is

$$\text{AI} = \frac{2 M K N}{\text{bytes moved}}, \quad (6)$$

where bytes moved counts the reads of X and $\widehat{W}$ and the write of Y . A configuration is compute-bound when its arithmetic intensity exceeds the hardware ridge point,

$$\text{AI}_{\text{ridge}} = \frac{\text{peak compute (FLOP/s)}}{\text{peak bandwidth (byte/s)}}, \quad (7)$$

and memory-bound otherwise. Byte-per-weight-element figures used in this analysis (1 byte for INT8 and ternary storage, 0.5 bytes for INT4) are approximations that do not account for auxiliary storage such as per-block quantization scales, and are reported as such. All experiments were run on a single NVIDIA RTX 2060 (6GB VRAM).

4 Results

4.1 Matrix-Size Crossover at Fixed Batch Size

At a moderate batch size ($M = 32$), the tensor-core ternary kernel outperformed both `bitsandbytes` baselines across small and medium matrix sizes, but lost to INT8 beyond a specific size threshold (Table 1).

Table 1: Execution time (ms) at $M = 32$.

$K \times N$	FP16	INT8	INT4	Ternary
128×128	0.023	0.264	0.284	0.103
768×3072	0.032	0.254	0.276	0.123
2048×2048	0.050	0.281	0.303	0.214
3584×3584	0.100	0.251	0.659	0.529
6144×6144	0.315	0.282	2.044	1.164

At this batch size, the ternary kernel beat INT8 at every size up to and including 2048×2048 , and lost from 3584×3584 onward, placing the crossover point against INT8 between approximately 4.2M and 12.8M weight elements. Against INT4, the ternary kernel won at every size tested in this range. Dense FP16 outperformed all quantized and ternary methods at every size tested.

4.2 Effect of Batch Size on the Crossover Point

The crossover point against INT8 was not fixed; it moved substantially earlier as batch size increased (Table 2, Figure 1).

Table 2: Approximate crossover region vs. INT8, by batch size.

M	Ternary beats INT8 up to	Crossover region
1	2048×2048	between 2048^2 and 3584^2
32	2048×2048	between 2048^2 and 3584^2
128	768×768	between 768^2 and 768×3072
256	768×768	between 768^2 and 768×3072

The crossover against INT8 held steady between $M = 1$ and $M = 32$, then shifted sharply earlier, by roughly an order of magnitude in matrix size, once M reached 128. The same pattern held against INT4.

This shift coincided directly with a change in the roofline classification of each configuration (Figure 2). At $M = 32$, every configuration tested, including the largest (6144×6144), remained memory-bound (arithmetic intensity below the GPU’s ridge point of approximately 76.79 FLOP/byte¹). At $M = 128$ and $M = 256$, configurations at 768×768 and above shifted to compute-bound. Once a configuration became compute-bound, raw floating-point throughput determined the outcome rather than memory traffic: the ternary kernel’s achieved throughput plateaued around 1,400–2,400 GFLOP/s across all compute-bound configurations, while INT8’s throughput scaled up to tens of thousands of GFLOP/s at the largest sizes.

4.3 Memory Footprint

Peak GPU memory usage did not follow the ordering that bit-width alone would suggest (Table 3).

INT4, nominally the smallest weight representation, consistently used the most memory of all four implementations, by a wide margin at larger sizes, and this gap grew with matrix size. The ternary kernel’s

¹GPU peak bandwidth (336 GB/s) and peak FP32-accumulate tensor-core throughput (25.8 TFLOP/s) figures used to compute this ridge point are commonly cited RTX 2060 specifications and should be confirmed against manufacturer documentation before final submission.

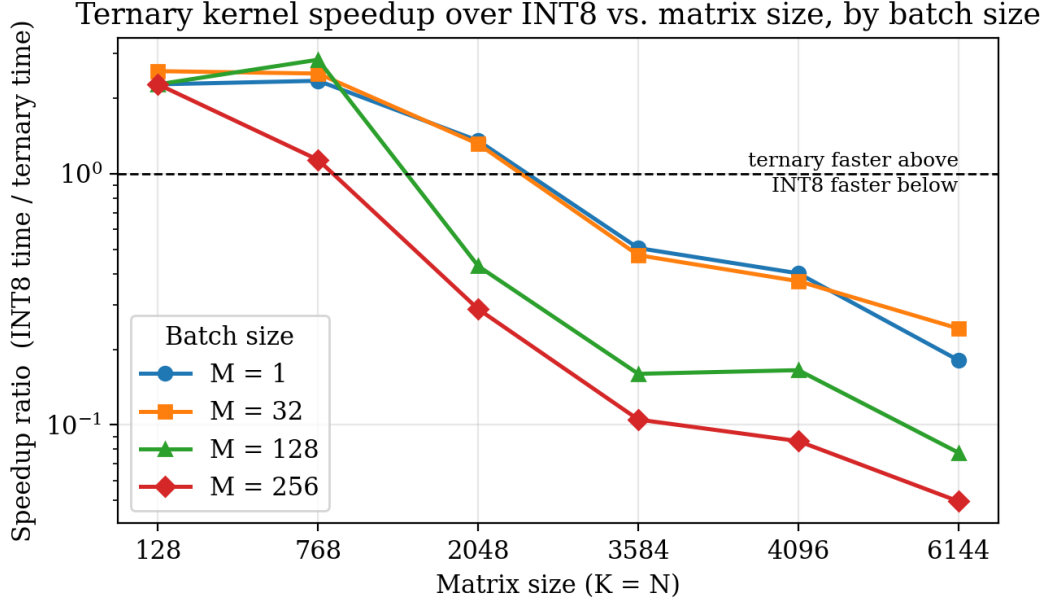


Figure 1: Ternary-vs-INT8 speedup ratio (INT8 time / ternary time) across matrix size, one line per batch size. Values above the dashed line at 1.0 indicate the ternary kernel is faster. The crossover moves to smaller matrix sizes as batch size increases.

Table 3: Peak memory usage (MB) at $M = 32$.

$K \times N$	FP16	INT8	INT4	Ternary
768×768	36.2	36.9	40.5	37.9
4096×4096	60.0	76.9	180.5	102.0
6144×6144	108.5	145.8	379.5	183.9

memory usage exceeded dense FP16 at every size tested, despite the underlying weight representation requiring less storage than FP16 weights.² This ordering held consistently across all four tested batch sizes.

4.4 Autotuning Result

An autotuned variant of the tensor-core ternary kernel, sweeping seven block-size and launch-parameter configurations via `triton.autotune`, converged to the identical configuration used in the manually tuned kernel (`BLOCK_M=BLOCK_N=BLOCK_K=32, num_warps=2, num_stages=2`) at every matrix size tested, including the largest (6144×6144). The autotuned wrapper measured 6–22% slower than the untuned kernel at equivalent configurations, attributable to autotune’s per-call dispatch overhead rather than any difference in the underlying computation, as confirmed by consistent, non-varying per-call timings across repeated calls at a fixed shape. No configuration among those tested, including substantially larger block sizes (up to 128×128 with 8 warps), improved on the original manual choice at any tested scale.

²One measurement ($M = 256, 768 \times 768$, ternary: 44.5 MB) fell below the implementation’s own value at the smaller 128×128 configuration and was treated as a caching-allocator artifact rather than a genuine result; this data point is excluded from the reported trend.

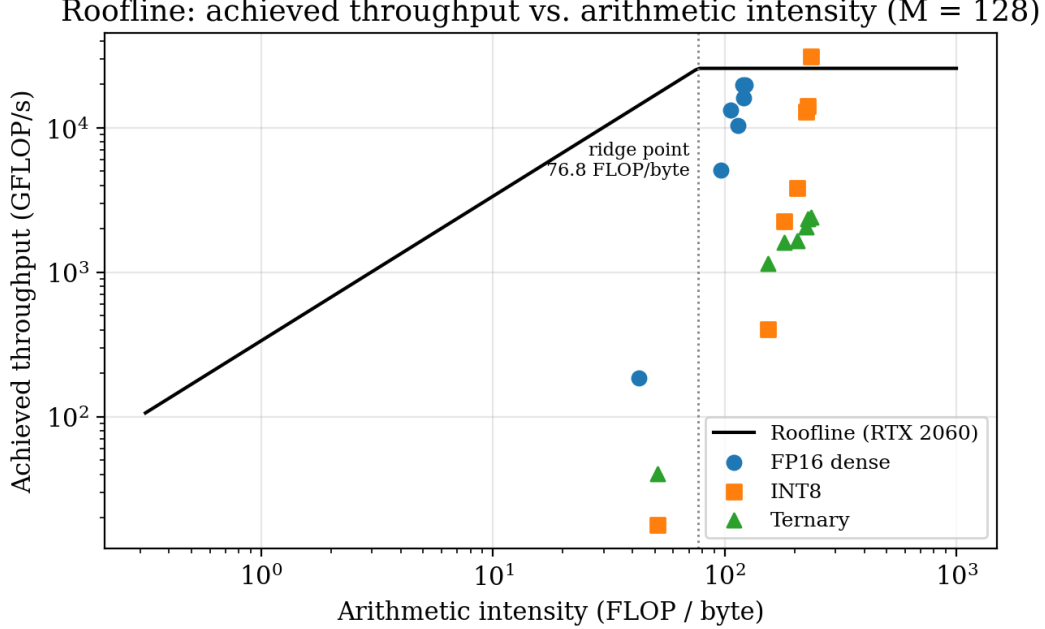


Figure 2: Roofline analysis at $M = 128$: achieved throughput versus arithmetic intensity. The ternary kernel’s throughput plateaus well below INT8 and FP16 in the compute-bound region (right of the ridge point), consistent with a fixed FP16 tensor-core ceiling.

5 Discussion

5.1 The crossover is governed by arithmetic intensity, not by whether multiplication is avoided

The central finding of this work is that the ternary kernel’s relative performance is well explained by its position relative to the GPU’s roofline ridge point, rather than by whether the kernel avoids multiplication. The crossover against INT8 moved by roughly an order of magnitude in matrix size as batch size increased, and this shift coincided with configurations moving from memory-bound to compute-bound. In the memory-bound regime, all implementations are limited primarily by how quickly weights can be streamed from memory, and a lower-precision representation (ternary, stored as 1 byte per element here) reduces that traffic relative to FP16 weights, which is sufficient to win regardless of how the multiply-accumulate step is expressed. Once a configuration becomes compute-bound, memory savings no longer matter, and raw throughput determines the outcome.

5.2 Why the compute-bound ternary kernel underperforms: an FP16 tensor-core ceiling

The selection-based (`tl.where`) kernel was substantially slower than the tensor-core (`tl.dot`) kernel at every matrix size, indicating that avoiding a multiply instruction in ordinary ALU code does not by itself yield a hardware benefit on this GPU. Modern GPUs execute multiply-accumulate as a single fused instruction through tensor cores; a kernel that avoids multiplication in scalar code does not skip a cycle relative to this path, it forfeits access to it.

This explains the selection-based kernel’s poor performance, but not fully why the tensor-core kernel itself underperforms INT8 once compute-bound. The likely explanation lies in the implementation: ternary weights are cast to FP16 before being passed to `tl.dot`, meaning the kernel executes at the FP16 tensor-

core throughput ceiling regardless of the fact that the underlying weight values require far less precision. The `bitsandbytes` INT8 path, by contrast, is able to use native INT8 tensor-core throughput, which is substantially higher than FP16 throughput on Turing-generation hardware. The achieved-throughput data in Figure 2 is consistent with this: the ternary kernel’s throughput plateaued in a narrow range across all compute-bound configurations, while INT8’s continued to scale. This is best read as a limitation of this specific implementation rather than a fundamental property of ternary computation; a kernel that computed directly in a low-bit representable format, rather than casting up to FP16, would not necessarily be subject to the same ceiling, though this was not implemented or tested here.

5.3 Memory footprint does not follow bit-width in a simple way

INT4 consistently used more peak memory than INT8, FP16, or the ternary kernel, despite requiring the least storage per weight. The most plausible explanation is that `Linear4bit` cannot compute directly on packed 4-bit values and must dequantize weights into a full intermediate buffer before the `matmul` executes, meaning INT4’s compactness at rest is not preserved at compute time. If accurate, this indicates that bit-width alone is an unreliable predictor of runtime memory footprint, and that a method’s dequantization strategy matters as much as its storage format. This explanation is consistent with the observed data but was not independently confirmed by instrumenting `bitsandbytes` internals, and is presented as the most likely explanation rather than a confirmed mechanism.

Separately, the ternary kernel’s own memory usage exceeded dense FP16 at every size tested. This is attributable to the kernel’s output computation, which creates multiple full-size intermediate tensors per call; this is an implementation inefficiency in the kernel used for this study, not evidence that ternary representations are inherently memory-costly at inference time.

5.4 The autotuning result suggests a structural ceiling

The autotuning investigation found that no tested configuration, including substantially larger block sizes, improved on the kernel’s original manually chosen parameters at any scale. This suggests that the performance gap between the tensor-core ternary kernel and cuBLAS is not attributable to poorly chosen block sizes or launch parameters, and likely reflects either the RTX 2060’s comparatively limited per-streaming-multiprocessor resources relative to larger configurations, or a deeper difference in tiling and memory-staging strategy between this kernel and cuBLAS’s implementation. Distinguishing between these would require lower-level profiling that was outside the scope of this study.

6 Limitations and Future Work

- **Quantization was applied post-hoc, without retraining.** The ternary weights carry substantial reconstruction error. This is appropriate for the kernel-level questions addressed here, which do not depend on model quality, but the results say nothing about model accuracy under ternary quantization; that is addressed by prior work [6, 4].
- **A single GPU was tested.** All results are specific to the RTX 2060. The crossover points are a function of this GPU’s peak bandwidth and compute throughput and should not be assumed to transfer to other hardware.
- **The tensor-core kernel casts weights to FP16** rather than computing natively in a lower-precision format, a likely contributor to its compute-bound underperformance. A kernel operating on packed low-bit representations directly was not implemented and is the most direct avenue for future improvement.

- **Bytes-moved figures are approximate**, omitting auxiliary storage such as per-block quantization scales.
- **One memory measurement was excluded as an anomaly** rather than explained; its root cause was not investigated.
- **INT8 execution time was largely size-invariant** across much of the tested range, which may indicate that fixed dispatch overhead within the `bitsandbytes` INT8 path dominates at these sizes. This was not isolated and should be treated as an open question when interpreting the precise crossover location, though it does not affect the qualitative finding.
- **The GPU peak specifications should be verified** against manufacturer documentation before the roofline classifications are treated as final.

Future work could implement a kernel that computes natively on packed low-bit representations, test across multiple consumer GPU generations, and evaluate whether the memory-bound regime identified here corresponds to real end-to-end inference conditions (e.g., single-token decoding at small batch sizes).

7 Conclusion

This paper investigated whether a custom ternary matrix multiplication kernel provides a measurable inference performance advantage over off-the-shelf low-bit inference methods on a consumer GPU, and under what conditions. The answer is conditional: a tensor-core-based ternary kernel outperformed `bitsandbytes` INT8 and INT4 in the memory-bound regime (small-to-medium matrix sizes at small batch sizes) but lost decisively once configurations became compute-bound, a transition that occurred at substantially smaller matrix sizes as batch size increased. This crossover is explained by arithmetic intensity relative to the GPU’s roofline ridge point rather than by whether multiplication is avoided in the kernel’s inner loop: a selection-based kernel that avoided multiplication entirely performed far worse than a tensor-core-based kernel that reintroduced it, indicating that access to fused multiply-accumulate hardware matters more than eliminating multiplication per se.

The central takeaway is that the premise motivating ternary kernels, that avoiding multiplication yields faster inference, is the wrong thing to measure. Across every configuration tested, performance was determined not by whether multiplication was avoided but by whether the operation was limited by memory bandwidth or by compute throughput. Ternary’s real advantage is that its smaller weights move less data, which helps only when data movement is the bottleneck; when compute is the bottleneck, that advantage is irrelevant and the kernel’s inability to use a native low-bit compute path makes it lose. This distinction has a direct practical consequence: the memory-bound regime in which ternary wins, small batches and single-token decoding, is precisely the regime in which consumer inference typically operates, whereas the compute-bound regime in which it loses reflects the large batched workloads more common to datacenter serving. A ternary kernel may therefore be most useful in exactly the setting this study found it to win, provided future work confirms the effect holds in an end-to-end decode loop rather than in isolated matrix multiplications. More generally, the result is a reminder that a low-level hardware advantage only translates into real speedup when it acts on the bottleneck that actually constrains the workload; identifying that bottleneck first is what determines whether the optimization matters at all.

References

- [1] Biderman, S., Schoelkopf, H., Anthony, Q., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202.
- [2] Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *arXiv preprint arXiv:2208.07339*.
- [3] Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314*.
- [4] Ma, S., Wang, H., Ma, L., et al. (2024). The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. *arXiv preprint arXiv:2402.17764*.
- [5] Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, 10–19.
- [6] Wang, H., Ma, S., Dong, L., et al. (2023). BitNet: Scaling 1-bit Transformers for Large Language Models. *arXiv preprint arXiv:2310.11453*.